MAHATMA GANDHI UNIVERSITY, KOTTAYAM
MGU-UGP (HONOURS) REGULAR EXAMINATION, October 2025 SCHEME FOR VALUATION

Part A – Short Answer Questions (Any 10 × 2 = 20 marks)

Each question carries **2 marks**.

Q. No	Expected Points	Marking Scheme	CO
1	**What is the input voltage for the power supply pins on a Raspberry Pi? What are the types of power output pins available on a Raspberry Pi?** The input voltage for the power supply pins on a Raspberry Pi is **5V.** The Raspberry Pi has several power output pins that provide different voltage levels: 1**. 5V Pins** or Pins 2 and 4 2. **3.3V Pins** or Pins 1 and 17	Each answer =1m	CO1
2	**What is the function of print() in Python programming?** The print() function in Python is used to output text or values to the screen. It is a built-in function that allows you to: 1. Display messages 2. Show variable values 3. Output results of calculations 4. Print data structures like lists, tuples, and dictionaries	Answer with function like this= 2m OR Function along with example also = 2m	CO1
3	**Can you list the main elements that make up the structure of a Python program?** A Python program typically consists of: 1. Import statements 2. Function definitions 3. Class definitions 4. Main program	Four elements = 2m If only 2 elements = 1m	CO1

4	**What will be the output of the following codes?** **1. x = 3** **y = 2** **print(x ** y)** **2. a = 23** **b = 4** **print(a // b)** 1. x ** y = $3^2 = 3 \times 3 = 9$ So, the output will be: 9 2. a // b = $23 \div 4 = 5.75 \rightarrow 5$ (integer part) So, the output will be: 5	Each answer =1m	CO1
5	**What are keywords in Python, and provide examples of keywords?** Keywords are reserved words in Python that have special meanings and are used to define the structure and syntax of the language. Examples of Keywords 1. *if* 2. *for* 3. *def* 4. *class* etc	Definition = 1m Example = 1m	CO1

6	**Briefly explain the requirements for setting up Raspberry Pi.** 1. Hardware Requirements - Raspberry Pi board, MicroSD card, Power supply, Display and Keyboard and mouse **2.** Software Requirements - Raspberry Pi OS and Imaging tool	1m for each requirement	CO1

7	**What does a tuple represent as a sequence in Python?** - Ordered, immutable collections of items that can be of any data type. Example : colors = ('red', 'green', 'blue') print(colors[0]) # Output: red	Long Explanation= 2m OR Simple Definition = 1m Example =1m	CO1

8	**Explain an example use of digital write in controlling an LED.**
	
	Here's an example of using digitalWrite to blink an LED connected to a digital pin:
	
	const int ledPin = 13;
	void setup()
	{
	pinMode(ledPin, OUTPUT);
	}
	void loop()
	{
	digitalWrite(ledPin, HIGH);
	delay(1000);
	digitalWrite(ledPin, LOW); //
	delay(1000);
	}
	In this example, we are:
	1. Setting up the pin: We set the pin mode to OUTPUT using pinMode.
	2. Turning the LED on and off: We use digitalWrite to set the pin HIGH and LOW, which turns the LED on and off.
		Code =2m	CO2

9	**What is the purpose of the int() function in Python? Explain with an example.** The int() function in Python is used to convert a value to an integer. It takes a number or a string as input and returns an integer value. Example: x = "10" y = int(x) print(y) # Output: 10	Explanation =1m Example = 1m	CO2

10	**Given the string = "Binary:101010", write a Phyton program to extract only the binary number from it.** You can use string slicing or splitting to extract the binary number. Method 1: Using Split string = "Binary:101010" binary_number = string.split(":")[1] print(binary_number) # Output: 101010 OR Method 2: Using Slicing string = "Binary:101010" binary_number = string[7:] # Start from index 7 to the end print(binary_number) # Output: 101010	Code= 2m	CO2
11	**Which Python library is used to play simple audio files?** The most commonly used library for playing simple audio files is Pygame.	Answer = 2m	CO2
12	**What are the fundamental steps in creating a Tkinter GUI?** To create a Tkinter Python app, follow these basic steps: 1. Import the tkinter module **2.** Create the main window (container) **3.** Set Window Properties **4.** Add widgets to the main window	4 steps = 2m	CO3

13	**Define pack in Tkinter geometry managers.** It organizes the widgets in blocks before placing in the parent widget. Widgets can be packed from the top, bottom, left or right. It can expand widgets to fill the available space or place them in a fixed size.	Definition = 2m	CO3
14	**What is the purpose of the label.place(x=50, y=50) line in the given code?** The label.place(x=50, y=50) line places the label at coordinates (x=50, y=50) within the window.	Answer = 2m	CO3

Part B – Short Essay Questions (Any 6 × 5 = 30 marks)

Each carries **5 marks**.

Q. No	Key Points Expected	Marking Scheme	CO
15	**What is Raspberry Pi, and what are its key features and applications?** **Name some popular models of Raspberry pi.** Raspberry Pi is a series of small, low-cost, and highly capable single board computers designed for a wide range of applications, from education to industrial automation. Features: (Minimum any 3 features) 1. RP2040 microcontroller chip: A dual-core ARM Cortex-M0+ processor-based chip designed for high-performance and low-power applications. 2. GPIO pins: Raspberry Pi boards feature GPIO) pins that allow users to connect and control external devices, such as sensors, motors, and LEDs. 3. Programming languages: Supported languages include MicroPython, CircuitPython, and C/C++. 4. Small size: Raspberry Pi boards are compact and can fit in small 5. Low cost: Raspberry Pi boards are affordable 6. High performance: Despite their small size and low cost, Raspberry Pi boards pack a significant amount of processing power and memory. Applications: Raspberry Pi boards are used in a wide range of applications, including: - Education, Home automation, Robotics, Media Centers.	Introduction = 1m Features = 2m Applications = 1m Models = 1m	CO1

	Some popular models include:(any 2 or 3 models) 1. Raspberry Pi Model B 2. Raspberry Pi Model A 3. Raspberry Pi Model B+ 4. Raspberry Pi 2 Model B 5. Raspberry Pi 3 Model B 6. Raspberry Pi 3 Model A+ 7. Raspberry Pi 3 Model B+		
16	**Explain the difference between integers, floats, and complex numbers in Python. Provide examples of each.** 1. int (Integer) - Represents whole numbers (positive, negative, or zero). Example: x = 10 y = -3 z = 0 print(type(x)) # <class 'int'> 2. float (Floating Point) - Represents real numbers (decimal values). Example: a = 3.14 b = -2.5 print(type(a)) # <class 'float'> 3. complex (Complex Numbers) - Used for complex numbers with a real and imaginary part. Example: c = 2 + 3j print(type(c)) print(c.real) print(c.imag) # <class 'complex'> # 2.0 # 3.0	Explanation =3m Example = 2m	CO1

17	**Explain how pin numbering affects hardware-software synchronization.** Pin numbering is crucial for hardware-software synchronization in embedded systems, robotics, and electronics projects. Here's why: 1. Pin identification: Pin numbering helps identify specific pins on a microcontroller, module, or circuit board. This ensures that software instructions are sent to the correct hardware pins. 2. Signal routing: Pin numbering determines how signals are routed between hardware components and the microcontroller. Incorrect pin numbering can lead to misconnected signals, causing system failures. 3. Software configuration: Pin numbering affects how software libraries and frameworks configure and interact with hardware peripherals. For example, a library might require specific pin numbers for SPI or I2C communication. 4. Hardware-software compatibility: Pin numbering inconsistencies can lead to compatibility issues between hardware and software components. Ensuring consistent pin numbering helps prevent these issues. 5. Efficient debugging and maintenance: Consistent and accurate pin numbering enables easier debugging and maintenance of complex systems. When pin numbering is correct, developers can quickly identify and isolate issues, reducing downtime and improving overall system reliability. This also facilitates collaboration among team members and simplifies the process of updating or modifying existing designs.	5 points with brief explanation = 5m	CO2
18	**Discuss the advantages and limitations of GPIO in interfacing external devices.** Advantages of GPIO: (any 3) 1. Flexibility: GPIO (General Purpose Input/Output) pins can be configured as either input or output, allowing for a wide range of applications. 2. Customizability: GPIO pins can be controlled and programmed to perform specific tasks, making them suitable for various projects and applications. 3. Low Cost: GPIO pins are often available on microcontrollers and single-board computers, making them a cost-effective solution for interfacing external devices. 4. Ease of Use: GPIO pins are relatively easy to use, especially with modern libraries and frameworks that provide simple APIs for controlling and interacting with	Advantages = 3m Limitations = 2m	CO2

	external devices. Limitations of GPIO: (any 2) 1. Limited Current Capacity: GPIO pins typically have limited current sourcing and sinking capabilities, which can restrict the types of devices that can be connected directly. 2. Voltage Limitations: GPIO pins often operate at specific voltage levels (e.g., 3.3V or 5V), which may not be compatible with all external devices. 3. Noise and Interference: GPIO signals can be susceptible to electrical noise and interference, which can affect the reliability and accuracy of data transmission. 4. Limited Number of Pins: The number of GPIO pins available on a microcontroller or single-board computer is typically limited, which can restrict the number of external devices that can be connected.		
19	**Discuss the role of GPIO libraries in modular and reusable code development.** (Any 5 points) 1. Abstraction: GPIO libraries abstract the low-level details of GPIO pin control, allowing developers to focus on high-level programming. 2. Modularity: GPIO libraries enable modular code development by providing a separate module for GPIO-related functionality. 3. Reusability: GPIO libraries can be reused across multiple projects, reducing code duplication and development time. 4. Readability: GPIO libraries often provide a simple and intuitive API, making code more readable and maintainable. 5. Portability: GPIO libraries can be ported to different platforms, allowing code to be easily migrated between devices. 6. Faster Development: GPIO libraries accelerate development by providing pre-written functions for GPIO operations. 7. Less Error-Prone: GPIO libraries reduce the likelihood of errors by handling low-level details and providing a standardized interface. 8. Easier Maintenance: GPIO libraries simplify maintenance by providing a single point of modification for GPIO-related changes.	5 points = 5m	CO2

20	**Give an overview of Tkinter 's functionality.** Tkinter is Python's built-in library for creating graphical user interfaces (GUIs). It acts as a lightweight wrapper around Tcl/Tk GUI toolkit, offering Python developers a simple and intuitive way to build desktop applications. It supports layout management, event handling and customization, making it ideal for rapid GUI development in Python. Tkinter Function: 1. Provides a built-in and easy-to-use way to create GUI (Graphical User Interface) applications in Python. 2. Offers various widgets like buttons, labels, text boxes and menus to build interactive apps. 3. Eliminates the need for external GUI frameworks tkinter comes bundled with Python. 4. Useful for building desktop applications like calculators, form apps and dashboards. 5. Supports event-driven programming, making it suitable for responsive user interfaces.	Introduction = 2m Function = 3m	CO3

21	**How can you customize the properties of Tkinter widgets?** There are a number of tkinter widgets which we can put in our tkinter application. Some of the major widgets are explained below: 1. Label - It refers to the display box where we display text or image. It can have various options like font, background, foreground, etc. 2. Button - A clickable button that can trigger an action. 3. Entry - It is used to input the single line text entry from the user. For multi-line text input, Text widget is used. 4.CheckButton - A checkbox can be toggled on or off. It can be linked to a variable to store its state. Customizing widget properties in Tkinter involves setting various options or attributes when creating a widget; 1. Setting Properties During Widget Creation: Many widget properties can be set directly as keyword arguments when you instantiate a widget. **2.** Modifying Properties After Widget Creation: You can change a widget's properties at any time	Any 3 widgets(introduction) = 3m 2 properties = 2m	CO3

	after it has been created using the config() or configure() method. These methods are interchangeable.		
22	**Can you explain the functionality of Tkinter widgets, with practical examples?** 1. Label It refers to the display box where we display text or image. It can have various options like font, background, foreground, etc. Syntax: w=Label(master, option=value) example: This code creates a simple GUI window using Tkinter that displays a text label. 2. Button A clickable button that can trigger an action. Syntax: w=Button(master, option=value) Example: This code creates a window titled "Counting Seconds" with a single "Stop" button. When button is clicked, window closes. 3 Entry It is used to input the single line text entry from the user.For multi-line text input, Text widget is used. Syntax: w=Entry(master, option=value) Example: This code creates a form with two labeled input fields: First Name and Last Name. 4.CheckButton A checkbox can be toggled on or off. It can be linked to a variable to store its state. Syntax: w = CheckButton(master, option=value) Example: This code creates a window with two checkboxes labeled "male" and "female".	Four widgets = 4m Example = 1m	CO3